Software Construction

Intro to LangChain

Ahmed Bahaj, Abdullah Misar, Yazeed Bafqeeh, Baraa Alnahari

About LangChain

LangChain is an open-source frameowrk designed to simplify the development of applications powered by large language models (LLMs). It provides a standardized interface to connect LLMs with external data sources, other tools and complex workflows.

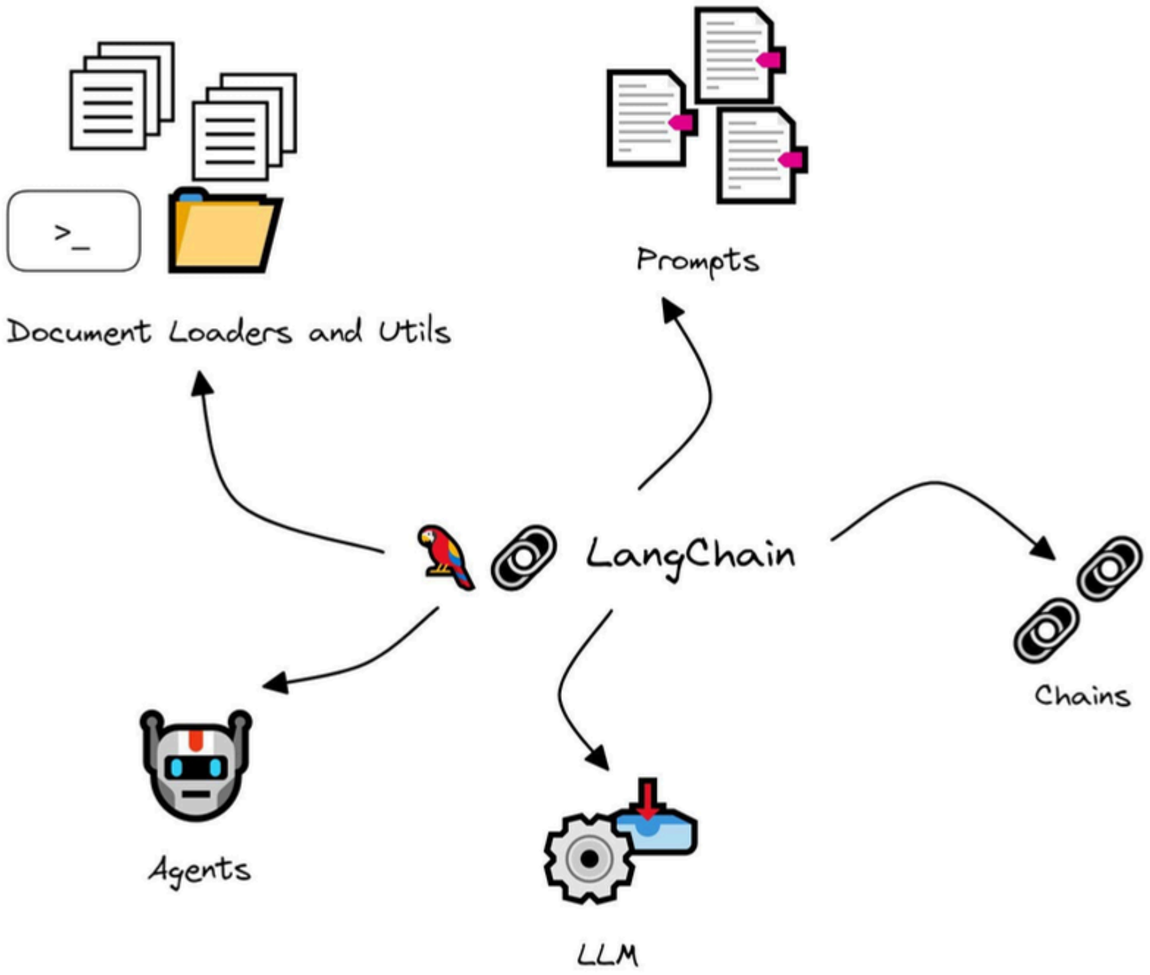
Purpose :

Enable developers to **build context-aware and reasoning** LLM applications by chaining togeher components

Target Users :

- ✓ Software Developers building AI applications
- ✓ ML engineers deploying production AI systems
- ✓ Researchers experimenting with LLM capabilities

Build Your own LLM!



Setup Guide

Dependency Install

```python
# Basic installation
pip install langchain

# With common dependencies
pip install langchain-openai langchain-community

# Full installation with all integrations
pip install langchain[all]
```

Environment Setup

```python
# Set up API keys (example for OpenAI)
import os
from dotenv import load_dotenv

load_dotenv()
os.environ["OPENAI_API_KEY"] = "your-api-key-here"
```

Core Concept: Category C

- Chains
- Prompts
- Vector Stores
- Embeddings
- Agents
- Tools
- Custom Tools

Core Concept: Chains

Chains are a sequence of calls to components (LLMs, tools, databases, documents) that **execute in order**. They allow you to combine multiple steps in a single workflow.

```python
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate

prompt = PromptTemplate(
    input_variables=["product"],
    template="What is a good name for a company that makes {product}?"
)

chain = LLMChain(llm=llm, prompt=prompt)
result = chain.run("Coffee Mugs")
```

Core Concept: Prompts

Templates that **guide** LLMs to generate specific outputs. **Langchain** provides tools for prompt engineering, templating and optimization.

Types

- Prompt templates
- Few-shot prompts
- Prompt selectors
- Output parsers

Core Concept: Embeddings

Numerical representations of text that capture semantic meaning. Used to convert documents into vectors for similarity search.

```
from langchain.embeddings import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()
vector = embeddings.embed_query("I am Ahmed, and I take software construction")
```

Core Concept: Vector Stores

- **Databases optimized for storing and retrieving** vector embeddings. Used for semantic search and Retrieval-Augmented Generation (RAG).

Embeddings

Chroma vector store
In-memory, lightweight, good for straightforward uses.

```python
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain.text_splitter import CharacterTextSplitter
from langchain_community.document_loaders import TextLoader

# Load and split documents
loader = TextLoader("ccsw325.txt")
documents = loader.load()
text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)
docs = text_splitter.split_documents(documents)

# Create embeddings
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")

# Create vector store
# Use Chroma, local and lightweight
vectorstore = Chroma.from_documents(
    documents=docs,
    embedding=embeddings
)

# Search
query = "What is software construction?"
results = vectorstore.similarity_search(query, k=3)

for doc in results:
    print(doc.page_content)
```

Core Concept: Agents

Entities that are **LLM-powered** that can use tools to perform actions and make decisions based on user-input.

```python
from langchain.agents import create_react_agent, AgentExecutor
from langchain_community.tools import CalculatorTool
from langchain_openai import ChatOpenAI
from langchain import hub

llm = ChatOpenAI(model="gpt-3.5-turbo")

# The agent can choose to use this tool if it detects a math problem
tools = [CalculatorTool()]

# Tells the LLM how to structure its thoughts and actions
prompt = hub.pull("hwchase17/react")

# Combines the LLM, the Tools, and the Prompt
agent = create_react_agent(llm, tools, prompt)

# Manages the loop: Thought -> Action -> Observation -> Final Answer
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# Note: We ask a math question. A normal LLM might guess;
# the Agent will decide to use the Calculator tool.
result = agent_executor.invoke({"input": "What is 534 multiplied by 23.5?"})

print(result['output'])
```

Prompt Template

Core Concept: Tools

Functions that agents can call to interact with the world (APIs, databases, calculators..etc)

Built-in tools:

- Wiki Search
- Google Search
- **SQL databse access**

SQL Agent

```python
from langchain_community.utilities import SQLDatabase
from langchain_community.agent_toolkits import create_sql_agent
from langchain_openai import ChatOpenAI

# Connect to a Database
db = SQLDatabase.from_uri("sqlite:///example.db")

llm = ChatOpenAI(model="gpt-4", temperature=0)

# Create a SQL Agent (pre-built for database queries)
agent_executor = create_sql_agent(llm, db=db, agent_type="openai-tools", verbose=True)

# Run a Natural Language Query
result = agent_executor.invoke({
    "input": "How many users are registered in the database?"
})

print(result["output"])
```

Core Concept: Custom Tools

In **LangChain**, you can leverage Python to create custom tools of your own!

In this example, we created a tool to fetch weather data using mock data.

You can replace the mock data with an API that fetches real-time data.

```python
from langchain.tools import tool
from langchain.agents import create_react_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain import hub
import requests

# Define a custom tool using the @tool decorator
@tool
def get_weather(city: str) -> str:
    """Get the current temperature for a given city (mock implementation)."""
    # Mock data
    mock_temps = {"London": "15°C", "Jeddah": "34°C", "New York": "18°C"}
    return mock_temps.get(city, "City not found")

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

# Register the custom tool
tools = [get_weather]

# Create Agent
prompt = hub.pull("hwchase17/react")
agent = create_react_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# Run the Agent
result = agent_executor.invoke({"input": "What's the weather in Jeddah?"})
print(result["output"])
```

Main Capabilities

Feature	Description
Modular Architecture	Plug-and-play customized components
RAG Support	Built-in support for retrieval-augmented generation
Memory Management	Conservation history and context tracking
Agent Framework	Tools for creating autonomus AI agents
Extensive Integrations	100+ integrations with LLMs, databases and APIs

Integrations

LLMs

- OpenAI
- Anthropic
- Hugging Face Models
- Llama
- ..etc

Vector Stores

- Chroma
- FAISS
- Pinecone
- Weaviate

Document Loaders

- PDF
- MS Word
- HTML
- Notion
- Github
- +80 sources

Constraints & Limitations

Steep Learning Curve
Could be an issue for non-technical, it requires time to master.

Performance Overhead
Chains are defintely slower than direct API calls

Cost Management
Easy to accidentally make many non-necessary LLM calls

Version Instability
Being open-source, rapid development can break existing code

Pricing

Langchain Core

- Open-source

FREE

LangSmith

- For Monitoring and debugging

$20 / month

LangServe

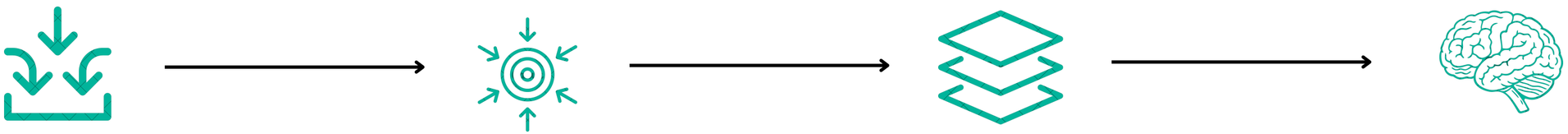
- Used for deployment

Free but hosintg costs

Comparison with Alternatives

Feature	LangChain	LlamaIndex	Haystack
Primary Focus	General LLM apps	RAG & Indexing	Search & QA
Learning Curve	Steep	Moderate	Moderate
Flexibility			
Documentation	Extensive	Good	Good
Community	Very Large	Growing	Established
When to favour	Complex Workflows	Simple RAG Applications	Enterprise use-cases

LangChain



1

User Input

2

Langchain Core

Chains, agents, tools

3

Components Layer

Prompts, memory, indexes, embeddings

4

Integrations Layer

LLMs, Vector Stores, APIs, document loaders

References

- Official Documentation: https://python.langchain.com/
- GitHub Repository: https://github.com/langchain-ai/langchain
- Community Discord: https://discord.gg/6adMQxSpJS
- LangSmith (Debugging): https://smith.langchain.com/